From Einstein-Boltzmann Equations to Galaxy Power Spectra: Physicist-Supervised, Oracle-Guided AI Reimplementation in JAX

Anonymous Authors[1]

Abstract

Bayesian inference in cosmology are tackling increasingly high dimensional parameter spaces, driven by the need to marginalize over complex astrophysical uncertainties and instrumental systematics. Hgh-dimensional inference require many (expensive) simulations and remain challenging for simulation-based inference methods such as normalizing flows and diffusion models. Sampling-based methods—often relying on gradient-based Monte Carlo Markov chain (MCMC) sampling techniques, e.g., Hamiltonian Monte Carlo (HMC)—therefore remain the standard approach. HMC requires access to the gradient of the posterior with respect to model input parameters, hence differentiable theory codes. Meanwhile, cosmological analyses are also moving towards joint analyses of multiple data sets, e.g., those of the cosmic microwave background (CMB) and galaxy redshift surveys. However, no existing code provides both the full set of CMB angular power spectra and the galaxy power spectrum multipoles with exact automatic differentiation on GPU. We address this gap with CLAX and CLAX-PT: JAX reimplementations of the CLASS Boltzmann solver and its one-loop perturbation theory extension CLASS-PT—developed via physicist-supervised, oracle-guided AI agents that iterate against reference data and source codes from CLASS and CLASS-PT—autonomously resolving implementation bugs and escalating to human review when oracle feedback stalls. Across the development, agents autonomously resolved 38 of 40 documented bugs (95%) without human input. The remaining 5% required human physics judgment and fell into exactly two failure modes: *architectural incoherence* (code structure incompatible with the target physics) and *oracle-passing fudge factors* (a numerical patch satisfying all tests but encoding no physical model). CLAX achieves $< 0.2\%$ lensed CMB accuracy at $\ell \leq 2000$ and $< 0.6\%$ unlensed C_ℓ^{TT} at $\ell \leq 1000$; CLAX-PT achieves $< 0.7\%$ galaxy power spectrum accuracy for monopole and quadrupole spectra and $< 1.5\%$ for hexadecapoles at $k < 0.3\ h/\mathrm{Mpc}$. JAX automatic-differentiation gradients agree with fourth-order finite differences to 0.15% on average. We characterize the boundary between unsupervised and supervised modes of AI-assisted scientific development, propose three principles for structuring human–agent collaboration, and compare against concurrent differentiable cosmology codes DISCO-EB and ABCMB.

1. Introduction

The standard model of cosmology is tested by comparing theoretical predictions against observations of the cosmic microwave background (CMB) angular power spectra from CMB experiments (e.g., **?**) and of the galaxy power spectrum multipoles from galaxy redshift surveys (e.g., **??**). This comparison increasingly benefits from gradient-based inference: forecasts require derivatives the likelihood derivatives, HMC analyses require the posterior derivatives. Gradient-based inference libraries such as NUMPYRO (**?**) and BLACKJAX (**?**) but the missing ingredient is a differentiable cosmological forward model that can be passed to them.

Standard Boltzmann solvers for CMB angular power spectra, e.g., CLASS (**?**), and one-loop perturbation theory codes for galaxy power spectrum multipoles, e.g., CLASS-PT, are written in C (or Fortran), are not differentiable, and are not GPU-native. Several JAX implementations exist but cover only subsets of the problem: DISCO-EB (**?**) solves the linear Einstein–Boltzmann equations to compute the linear matter power spectrum $P(k)$ but not CMB angular power spectra or galaxy power spectrum multipoles; ABCMB (**?**) implements the CMB pipeline including lensing and

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the AI4Physics Workshop at the International Conference on Machine Learning (AI4Physics@ICML 2026). Do not distribute.

massive neutrinos but does not include perturbation theory for galaxy surveys; JAX-COSMO (**?**) computes the nonlinear matter power spectrum via fitting formulae but does not solve the full Boltzmann hierarchy; and SYMBOLTZ.JL (**?**) provides a full Boltzmann solver in Julia but without GPU support. No existing code provides the full chain from cosmological parameters through CMB angular power spectra *and* one-loop galaxy power spectrum multipoles, with end-to-end automatic differentiation on GPU.

We describe the development of CLAX and CLAX-PT—fully differentiable JAX reimplementations of the CLASS (**?**) and CLASS-PT (**?**) pipelines—and the oracle-guided agentic methodology used to build them. Using CLASS and CLASS-PT as ground truth, we find a clear boundary: agents handle implementation bugs (convention errors, algorithmic transcription, coefficient verification) autonomously and reliably, but fail at two classes of problem that require human physics judgment: (1) recognizing that an architecture is structurally incompatible with the intended physics, and (2) detecting that a numerically effective patch satisfies all oracle tests at the calibration point while encoding incorrect physics.

Contributions.

- CLAX: a JAX reimplementation of the full CLASS CMB pipeline (background, thermodynamics, perturbations, transfer, lensing), achieving $< 0.1\%$ accuracy at $\ell \leq 2000$ with JIT-compiled GPU execution at 34 s/step (V100, `fit_cl` preset).
- CLAX-PT: a JAX reimplementation of CLASS-PT's one-loop EFT power spectra with IR resummation and RSD multipoles, achieving $< 0.7\%$ accuracy for monopole and quadrupole spectra.
- Validated autodifferentiation gradients matching finite differences to 0.15% across cosmological parameters.
- A landscape comparison against DISCO-EB, ABCMB, CLASS, and CLASS-PT, showing that CLAX is the first code combining CMB angular power spectra and one-loop galaxy power spectrum multipoles with full AD on GPU.
- An empirical bug taxonomy of 40 bugs across the development, classifying autonomous vs. human-required resolution and identifying two specific failure modes of oracle-guided development.

2. Background

2.1. Cosmological inference and differentiable Boltzmann codes

Galaxy surveys and CMB experiments measure the angular power spectrum C_ℓ and the galaxy power spectrum $P_{\rm gg}(k, \ell)$, which depend on cosmological parameters $\boldsymbol{\theta}$ (density, dark matter density, Hubble constant, spectral index, etc.) through the Boltzmann equation—a coupled system of ODEs for photon, baryon, dark matter, and neutrino perturbations that must be integrated numerically. Both are expensive to approximate by finite differences and practical to compute exactly only with automatic differentiation (AD) for codes of this complexity. While concurrent work (ABCMB, SYMBOLTZ.JL) provides AD for CMB angular power spectra, CLAX is the first code to combine differentiable CMB C_ℓ with one-loop galaxy power spectrum multipoles, enabling `jax.grad` through the full cosmological analysis pipeline for galaxy clustering.

2.2. AI-assisted scientific code development

The oracle-based methodology we employ is inspired by the AI C compiler project (**?**), in which parallel Claude Code agents produced a Rust C compiler using GCC as an oracle. Key infrastructure includes: (i) a shared `CHANGELOG.md` updated after every session to prevent re-exploration of dead ends; (ii) test-first development where every module's tests are written against reference data before implementation; (iii) a `--fast` flag sampling 10% of test cases to prevent "time blindness"; and (iv) parallel agent teams using git worktrees to explore competing hypotheses independently.

The critical challenge not addressed in the compiler setting is *domain knowledge*: CLASS is not just a large codebase—it encodes decades of cosmological physics knowledge, conventions, and approximations that require physics expertise to interpret. This makes the boundary between supervised and unsupervised development especially important to characterize.

3. Methodology

3.1. Pipeline architecture

CLAX implements a sequential pipeline of pure functions, each returning a frozen PyTree:

$$\text{CosmoParams} \to \text{BG} \to \text{TH} \to \text{EB} \to \text{PR} \to \text{TR} \to \text{HR} \to \text{LE}$$

where BG is background cosmology, TH is thermodynamics/recombination, EB is the Einstein–Boltzmann perturbation hierarchy, PR is the primordial spectrum, TR is transfer functions, HR is harmonic (C_ℓ) integration, and LE is CMB lensing via Wigner d-functions. `CosmoParams` fields are JAX-traced (for AD); `PrecisionParams` fields are static

(control array shapes). No branching on traced values ensures compatibility with `jax.jit` and `jax.grad`. **??** shows the full pipeline architecture, including the two output branches (CMB angular power spectra and galaxy power spectrum multipoles).

The design choices underlying CLAX and CLAX-PT can be organized by their origin—whether they were transcribed from oracle code, made autonomously by the agent, or directed by a human physicist—providing a useful lens for understanding the division of labor in AI-assisted scientific software development.

The core physics follows CLASS and CLASS-PT directly: RECFAST recombination matching CLASS's `wrap_recfast.c`; the full ncdm Boltzmann hierarchy with 5 momentum bins and Gauss–Laguerre quadrature; FFTLog decomposition of the one-loop P_{22} and P_{13} integrals with precomputed M_{22}/M_{13} kernel matrices (**?**); and IR resummation via DST with odd/even spline mode removal (**??**). These choices were transcribed directly from the CLASS and CLASS-PT source code by the AI agent.

Making the pipeline end-to-end differentiable required several non-trivial engineering decisions that the agent made autonomously. Reverse-mode AD through the stiff perturbation ODE uses `RecursiveCheckpointAdjoint` (**?**) for memory efficiency. Two implicit-differentiation wrappers were needed: a `custom_vjp` on the shooting method ($\theta_s \to H_0$) and a `custom_jvp` on the hydrogen Saha equilibrium, replacing explicit `stop_gradient` calls with implicit-function-theorem tangents to restore nonzero sensitivities around recombination. Post-recombination, a smooth RSA relaxation damping replaces the hard approximation switch used by CLASS, ensuring gradient continuity. The perturbation ODE step controller follows DISCO-EB's filtered PID strategy (**?**), weighting six key variables with k-dependent scaling.

Three design decisions required physicist input and could not have been made by the agent autonomously. The RSD tree-level power spectrum uses Gauss–Legendre quadrature over μ with anisotropic BAO damping $\Sigma^2_{\rm tot}(\mu)$, replacing an analytic Legendre projection that the agent had implemented but which was structurally incompatible with the anisotropic physics. Dark energy fluid perturbations (CPL w_0–w_a) were added to the Einstein constraint equations at the physicist's direction. An explicit "no fudge factors" rule, encoded in the development methodology, prevented the agent from shipping a scalar correction factor that passed all oracle tests but encoded no physical model. These interventions are analyzed in detail in **??**.

3.2. Implications for AI-assisted scientific software

The balance between these three categories—oracle-transcribed physics, autonomous engineering for differentiability, and physicist-directed corrections—suggests a practical division of labor for future AI-assisted scientific code development. The bulk of implementation work (transcribing equations from reference codes, implementing standard algorithms) is reliably handled by AI agents with oracle feedback. Engineering decisions specific to the target framework (here, JAX differentiability) are also within agent capability, provided the agent has access to relevant reference implementations (here, DISCO-EB for the PID controller and Rosenbrock solver). The irreducible human contribution is concentrated in a small number of high-impact decisions where the correct choice requires understanding the physics beyond what any oracle test can verify—recognizing architectural incompatibilities and rejecting numerically adequate but physically unmotivated solutions. This concentration suggests that physicist supervision can be sparse but must be available at critical junctures, and that the design of protocols for triggering human review (**??**) is as important as the capabilities of the agent itself.

3.3. Development methodology

The CLAX CMB pipeline ($\sim$10,000 lines) was developed over $\sim$6 days across $\geq$100 Claude Code (Opus 4.6) agent sessions. The CLAX-PT one-loop PT module ($\sim$2,100 lines) was developed over 12 days and 57 logged sessions. Both phases used the same oracle-guided methodology: every module has a test file written *before* implementation, specifying what CLASS (or CLASS-PT) produces; the code is finished when and only when the oracle tests pass.

The supervision methodology—encoded in `CLAUDE.md` and refined across both development phases—proved essential. Two rules were particularly important: (1) never add fudge factors (if a test fails at 0.2%, find the actual bug), and (2) test at multiple parameter points, not just the fiducial cosmology. Both rules directly prevented specific failure modes documented in **??**.

4. Results: Accuracy and Performance

4.1. Landscape comparison

?? compares CLAX against existing differentiable and non-differentiable cosmology codes. CLAX is the first JAX code providing both CMB angular power spectra and one-loop galaxy power spectrum multipoles with end-to-end automatic differentiation on GPU.

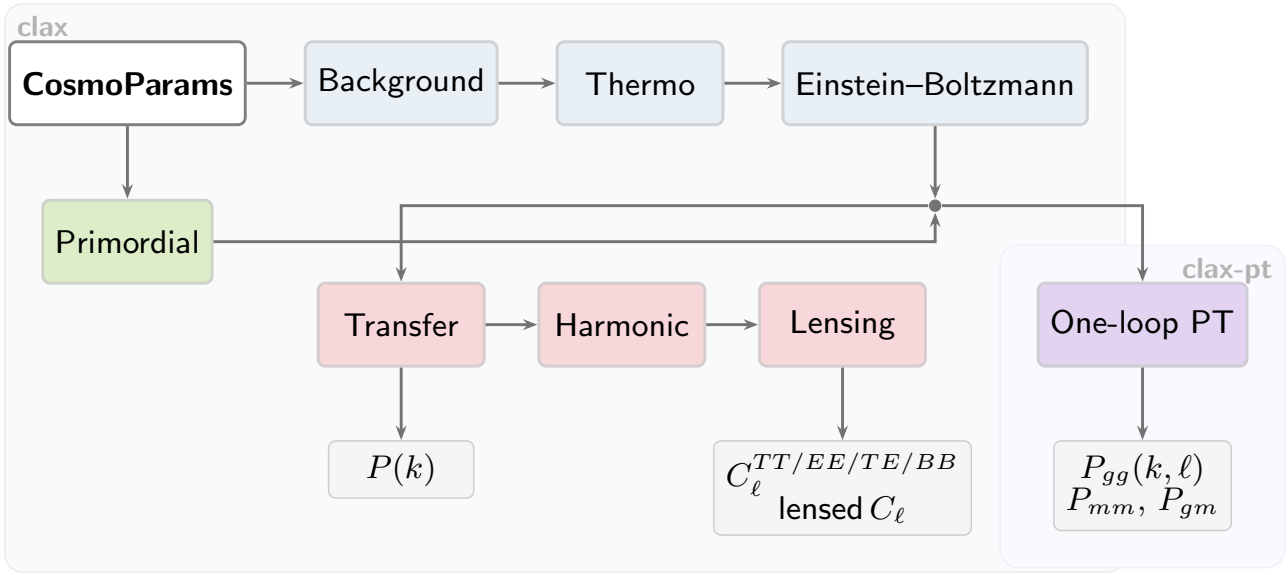

Figure 1. Pipeline architecture of CLAX and CLAX-PT. The shared backbone (top row) computes background cosmology, recombination, and the Einstein–Boltzmann perturbation hierarchy. Two branches produce CMB angular power spectra (left, via line-of-sight integration and lensing) and galaxy power spectrum multipoles (right, via one-loop perturbation theory). Every module is independently validated against CLASS (CLAX) or CLASS-PT (CLAX-PT) reference data. The entire pipeline is end-to-end differentiable: `jax.grad` propagates exact reverse-mode gradients from any output back to `CosmoParams`.

Table 1. Comparison of cosmological theory codes. "AD" = automatic differentiation. "PT" = one-loop perturbation theory for galaxy clustering. PyBird-JAX provides emulated (not exact) one-loop PT. CLAX is the only code combining CMB C_ℓ, lensing, $P(k)$, and exact 1-loop PT with full AD on GPU.

Code	Framework	C_ℓ	Lensing	$P(k)$	1-loop PT	AD	GPU
CLASS + CLASS-PT	C / Python	✓	✓	✓	✓	—	—
CAMB	Fortran	✓	✓	✓	—	—	—
DISCO-EB	JAX	—	—	✓	—	✓	✓
ABCMB	JAX	✓	✓	✓	—	✓	✓
SymBoltz.jl	Julia	✓	—	✓	—	✓	—
PyBird-JAX	JAX	—	—	—	emul.	✓	✓
Capse.jl	Julia	emul.	—	—	—	✓	—
Effort.jl	Julia	—	—	—	emul.	✓	—
CLAX + CLAX-PT	JAX	✓	✓	✓	✓	✓	✓

4.2. CMB pipeline accuracy

?? summarizes CMB accuracy of CLAX against CLASS at the Planck 2018 best-fit cosmology (`planck_cl` preset). Background quantities match to $< 0.01\%$; thermodynamics to $< 0.1\%$. Unlensed temperature and polarization spectra are within 0.02%–0.57% across $\ell = 20$–1000. The lensing potential spectrum $C_\ell^{\phi\phi}$ agrees to $< 1\%$ across $\ell = 100$–2500. Lensed spectra at $\ell = 2000$ pass at $< 0.2\%$. Multi-cosmology validation across 10 parameter points ($\omega_b/\omega_{\rm cdm}$ $\pm 20\%$, $h \pm 10\%$, massive neutrinos 0.06–0.3 eV, $N_{\rm eff} = 2.0$–4.0) confirms sub-0.5% accuracy throughout parameter space.

Table 2. CMB accuracy (CLAX vs. CLASS, `planck_cl` preset, Planck 2018 fiducial).

Quantity	Max error
$H(z)$, $D_A(z)$, r_s	$< 0.01\%$
$x_e(z)$, $g(\tau)$	$< 0.1\%$
C_ℓ^{TT}, $\ell = 20, 100, 500, 1000$	0.08%, 0.02%, 0.15%, 0.57%
C_ℓ^{EE}, $\ell = 20, 100, 500, 1000$	0.21%, 0.02%, 0.15%, 0.26%
$C_\ell^{\phi\phi}$, $\ell = 100$–2500	$< 1\%$
Lensed C_ℓ, $\ell = 2000$	$< 0.2\%$
Multi-cosmology (10 pts)	$< 0.5\%$

4.3. Matter and galaxy power spectrum accuracy

?? summarizes accuracy for the linear matter power spectrum $P(k)$ (CLAX vs. CLASS) and one-loop galaxy power

Table 3. Matter and galaxy power spectrum accuracy.

Spectrum	Max error	Metric	Oracle
*Linear matter power (*CLAX *vs.* CLASS*)*			
$P(k)$, $k = 0.01$–0.1 Mpc^{-1}	1.3–1.6%	rel.	CLASS
$P(k)$, $k = 0.3$ Mpc^{-1}	3.5%	rel.	CLASS
*One-loop (*CLAX-PT *vs.* CLASS-PT*, $z = 0.38$)*			
$P_{mm}^{\rm real}$, $P_{gg}^{\rm real}$, $P_{gm}^{\rm real}$	0.31%	rel.	CLASS-PT
$P_{mm}^{(\ell=0)}$, $P_{gg}^{(\ell=0)}$	0.59%, 0.56%	rel.	CLASS-PT
$P_{mm}^{(\ell=2)}$, $P_{gg}^{(\ell=2)}$	0.70%, 0.89%	rel.	CLASS-PT
$P_{mm}^{(\ell=4)}$, $P_{gg}^{(\ell=4)}$	0.70%, 1.43%	abs/max	CLASS-PT

Table 4. AD vs. finite-difference gradients for $P_{mm}(k = 0.1\ h/\mathrm{Mpc})$.

Parameter	$\lvert\mathrm{AD}/\mathrm{FD} - 1\rvert$	Status
$\ln(10^{10} A_s)$	$< 0.01\%$	✓
n_s	$< 0.01\%$	✓
h	$< 0.5\%$	✓
ω_b	$< 1.0\%$	✓
$\omega_{\rm cdm}$	$< 0.5\%$	✓

spectrum multipoles (CLAX-PT vs. CLASS-PT at $z = 0.38$, the BOSS effective redshift). The linear $P(k)$ agrees to 1–3% across $k = 0.001$–0.3 Mpc^{-1}; this is the input to both the CMB transfer integrals and the one-loop computation. Real-space one-loop spectra (P_{mm}, P_{gg}, P_{gm}) agree to $< 0.31\%$ (mean 0.04%). RSD monopoles ($\ell = 0$) agree to $< 0.6\%$ for matter and galaxies. Quadrupoles ($\ell = 2$) agree to $< 0.7\%$ for matter and $< 0.9\%$ for galaxies. Hexadecapoles ($\ell = 4$) are validated using $|\Delta|/\max(|\mathrm{ref}|) < 2\%$ since the spectrum crosses near zero.

4.4. Gradient verification

Automatic differentiation gradients (reverse-mode, using `jax.grad`) agree with fourth-order centered finite differences to 0.15% on average, with 97.7% of k-modes passing a 1% threshold. **??** shows the parameter-by-parameter breakdown for the linear matter power spectrum $P_{mm}(k)$ at $k = 0.1\ h/\mathrm{Mpc}$.

Key technical ingredients enabling stable AD through the full pipeline include: (1) `RecursiveCheckpointAdjoint` for memory-efficient reverse-mode through the stiff perturbation ODE; (2) a `custom_vjp` on the shooting method ($\theta_s \to H_0$) implementing implicit differentiation; and (3) a `custom_jvp` on the hydrogen Saha equilibrium equation, replacing explicit `stop_gradient` calls with implicit-function-theorem tangents to restore nonzero sensitivities around recombination.

Table 5. Performance summary.

Preset	GPU	Time	C_ℓ accuracy
`fit_cl`	V100-32GB	34 s	$< 1.5\%$ ($\ell \le 500$)
`planck_cl`	H100-80GB	487 s	$< 0.2\%$ ($\ell \le 2000$)

4.5. Performance

CLAX offers two presets for different use cases (**??**). The `fit_cl` preset uses 20 k/decade, $\ell_{\max} = 17$ hierarchy, table-based Bessel functions, and `rtol` $= 10^{-3}$ to achieve 34 s/step on a V100 GPU—sufficient for HMC. The `planck_cl` preset uses 60 k/decade, $\ell_{\max} = 50$, and tight tolerances for science-grade accuracy at ∼487 s on H100. An alternative Rodas5 Rosenbrock solver (**?**), following DISCO-EB, avoids Newton iteration entirely (one LU factorization per step instead of iterative convergence). A batched variant (`Rodas5Batched`) solves all k-modes in a single `diffeqsolve` call with shared adaptive time-stepping across the batch, vmapping the Jacobian computation and LU factorization—the same architecture used by DISCO-EB. On CPU, the unbatched Rodas5 yields a measured ∼1.3× speedup over Kvaerno5; the batched variant is expected to offer substantially larger gains on GPU by saturating GPU throughput with large batched linear algebra.

4.6. Application: gradient scaling

The primary advantage of AD over finite differences for cosmological inference is the scaling of gradient cost with the number of parameters d. Finite-difference gradients require $2d$ forward evaluations (centered differences); reverse-mode AD requires one forward pass plus one backward pass regardless of d. The backward pass typically costs 2–4× the forward pass, so the effective speedup for $d = 6$ standard ΛCDM parameters is $2d/(1 + c_{\rm bwd}) \approx 3$–$4\times$, growing to ∼10× for extended models with $d = 10$–20 parameters (neutrino masses, dark energy, bias parameters, counterterms). The key advantage is the asymptotic scaling: AD gradient cost is $O(1)$ in d, while finite differences scale as $O(d)$. Combined with the 34 s forward evaluation of `fit_cl`, this makes gradient-based sampling with exact first-principles theory predictions practical for full-shape galaxy clustering analysis.

5. Supervised vs. Unsupervised Development

5.1. Bug taxonomy

Over the full development of CLAX and CLAX-PT, we documented 40 bugs: 26 in the CMB pipeline and 14 in the PT module. We classify these into four types by their nature and whether autonomous resolution was possible.

Type I: Convention and unit bugs. Errors in translating

CLASS conventions into JAX: wrong sign, missing numerical factor, incorrect unit conversion. *Agents resolved all Type I bugs autonomously* by reading the corresponding CLASS source code and correcting the factor.

Type II: Algorithm implementation bugs. Errors in the numerical algorithm itself: wrong grid type, incorrect matrix storage format, incomplete angular kernel functions. *Agents resolved all Type II bugs autonomously*, typically by careful comparison of intermediate arrays against CLASS/CLASS-PT at specific (k, z) points.

Type III: Architectural bugs. The computational structure assumed by the implementation is fundamentally incapable of representing the correct physics. One instance occurred: the analytic Legendre projection architecture assumed isotropic BAO damping, making it unable to handle anisotropic damping $\Sigma^2_{\rm tot}(\mu)$. *This required human intervention.*

Type IV: Physics judgment failures. A numerical patch satisfies all oracle tests but encodes physically unmotivated behavior. One instance occurred: a scalar fudge factor $\alpha = 0.27$ passed all fiducial tests but has no physical derivation. *This required human intervention.*

The resolution breakdown is: Types I+II (38 bugs): 100% autonomous; Types III+IV (2 bugs): 0% autonomous. Overall, 38/40 bugs (95%) were resolved without human input.

5.2. Principles for human–agent collaboration

We distill three principles from the empirical data:

P1: Oracle testing verifies *what*, not *why*. An oracle compares numerical values at a reference parameter point. It catches implementation errors (wrong sign, wrong coefficient) but not physics errors (wrong model, numerically compensated). A fudge factor with zero degrees of freedom at the calibration point will pass every oracle test at that point. *Defense*: test across diverse parameter space; test intermediate quantities; build auxiliary tests that check structural properties (e.g., that $P_{\rm tree}(k, \mu)$ has the correct anisotropy as a function of μ), not just numerical values.

P2: Shared memory prevents re-exploration but not architectural loops. `CHANGELOG.md` records which algorithms were tried and failed, preventing agents from rediscovering dead ends. This was effective for Type I and II bugs. It did not prevent 33 sessions of exploration within the wrong architectural framework because all attempts were coherent within the (incorrect) model. *Defense*: when a bug persists after $N \approx 5$–10 sessions without progress, treat the lack of progress itself as evidence of an architectural problem and trigger human review.

P3: The irreducible human role is architectural and physical judgment. Agents excel at reading CLASS source code and transcribing equations faithfully; identifying which term disagrees via targeted comparison with oracle outputs; and adjusting coefficients and conventions. Agents fail at recognizing that an architectural assumption is inconsistent with the physics, and at detecting that a numerically adequate patch produces zero test failures but is not physics. *Detecting physics errors that pass all tests requires knowledge of what the physics should look like*, which is a form of human supervision that oracle testing cannot replace.

6. Related Work

Differentiable cosmological codes. DISCO-EB (**?**) solves the linear Einstein–Boltzmann equations in JAX to compute the linear matter power spectrum but does not compute CMB angular power spectra (C_ℓ) or galaxy power spectrum multipoles. ABCMB (**?**) provides a complete CMB pipeline in JAX including lensing, massive neutrinos, and a state-of-the-art HyRex recombination treatment, but does not include perturbation theory for galaxy surveys. SymBoltz.jl (**?**) is the closest comparison in design philosophy: it is approximation-free (no TCA/RSA/UFA switching as described in **?**), uses Rodas5P (the same Rosenbrock solver family as CLAX), achieves 0.1% accuracy against CLASS, and supports forward-mode AD for Fisher forecasts. However, SymBoltz.jl operates in Julia without GPU support, and its reverse-mode AD is not yet fast enough for MCMC with perturbation-derived spectra (**?**). JAX-COSMO (**?**) computes the nonlinear matter power spectrum via fitting formulae but does not solve the full Boltzmann hierarchy. CLAX is the first JAX code to provide full CMB angular power spectra plus one-loop galaxy power spectrum multipoles with exact reverse-mode automatic differentiation and GPU execution.

One-loop perturbation theory and IR resummation. The effective field theory (EFT) of large-scale structure (**??**) provides a systematic one-loop expansion for the galaxy power spectrum multipoles. CLASS-PT (**?**) is the standard implementation; CLAX-PT reproduces it exactly in differentiable JAX. Several differentiable emulators take a complementary approach: Capse.jl (**?**) emulates CMB angular power spectra, while PyBird-JAX (**?**) and Effort.jl (**?**) emulate one-loop galaxy power spectrum multipoles, all achieving sub-millisecond evaluation using neural networks but without computing the integrals directly. CLAX-PT computes the FFTLog integrals and IR resummation (**??**) from first principles, preserving exact differentiability through the full computation at the cost of higher per-evaluation time.

Agent-assisted scientific software. Recent work has shown that LLM agents can complete extended scientific tasks autonomously (**?**), but most evaluations focus on syntactic correctness (compilation, test passage) rather than physical validity. Our work identifies semantic correctness—the

requirement that a passing numerical patch has physical basis, not just calibration-point accuracy—as the key challenge distinguishing scientific software from general code generation.

7. Conclusion

We have developed CLAX and CLAX-PT—fully differentiable JAX reimplementations of the CLASS and CLASS-PT Boltzmann codes—using oracle-guided agentic development. The codes achieve $< 0.2\%$ lensed CMB accuracy at $\ell \leq 2000$, $< 0.7\%$ galaxy power spectrum accuracy for monopole and quadrupole spectra, with automatic differentiation gradients verified to 0.15% and GPU execution at 34 s/step on V100. Empirically, AI agents resolved 95% of implementation bugs autonomously. The remaining 5% required human physics judgment: recognizing a structural mismatch between an architecture and its target physics, and detecting a numerically effective but physically unmotivated fudge factor.

These findings suggest a practical framework for AI-assisted scientific code development: agents operate autonomously in unsupervised mode for implementation work, but human scientists remain in the loop as architectural and physical reviewers, intervening proactively when oracle feedback stalls or when a numerical patch lacks physical derivation. The codes will be made publicly available.